

TRACE-HEP

GUI-First Documentation - v0.1.19 - Toolkit for
Rendering and Analysis of Collider Events

What is TRACE-HEP?

- * A format-agnostic Python library for reconstructed-level collider event and vertex displays
 - * trace-gui: a live interactive viewer -- the primary, day-to-day way to use it
 - * Everything the GUI does is also one function call or one CLI command away, for scripting and batch jobs (shown alongside each screenshot in this deck)
- * Polar (ϕ , $|\eta|$), beam-axis 2D, and interactive 3D event views; a companion vertex-display family for pileup
- * Loaders for Delphes, flat-ntuple, calo-timing, and public ATLAS/CMS Open Data ROOT files -- same loaders, GUI or script
- * A browsable failure-mode/anomaly gallery, with built-in auto-flagging by 11 standard HEP criteria
 - * Core package has zero ROOT/uproot dependency; open source, MIT licensed

Where to find it

GitHub: github.com/wasikatcern/TRACE-HEP
TestPyPI: test.pypi.org/project/trace-hep
License: MIT

Install and launch the GUI

```
python3 -m venv .venv
source .venv/bin/activate

pip install --index-url https://test.pypi.org/simple/ \
--extra-index-url https://pypi.org/simple/ \
"trace-hep[gui,delphes]"

trace-gui
# -> TRACE viewer running at http://127.0.0.1:5057/
#      (opens automatically in your browser)
```

"[gui]" adds Flask, "[delphes]" adds uproot/awkward for ROOT files. Core alone (no extras) is just numpy, matplotlib, and plotly.

Or use it as a library / from the shell

```
# Python
python3 -c "import tracehep as trace; print(trace.__version__)"

# command line, no Python needed
trace-batch --input file.root --indices 0 1 2 --outdir out/ \
--which polar beam2d --show-tracks

# from source (editable, for developing tracehep itself)
git clone https://github.com/wasikatcern/TRACE-HEP.git && cd TRACE-HEP
pip install -e ".[dev]" && pytest -q
```

Every GUI feature in this deck has a scripted or CLI equivalent -- shown as ">>> ..." under each screenshot below.

The data model

```
    Jet(pt, eta, phi, mass=0, btag=False, is_hs=None)
    Track(pt, eta, phi, charge=0, d0=0, z0=0, x=0, y=0, z=0, ...)
Lepton(pt, eta, phi, flavor)    Photon(pt, eta, phi)    MissingET(pt, phi)

    Event(jets, tracks, muons, electrons, photons, met, ...)
    VertexEvent(vertices, tracks, truth_vertices, mu=None, ...)
```

Nothing here knows about ROOT or any ntuple format -- build these by hand, or load them from Delphes / flat-ntuple / Open Data files; every display and the GUI work unchanged either way.

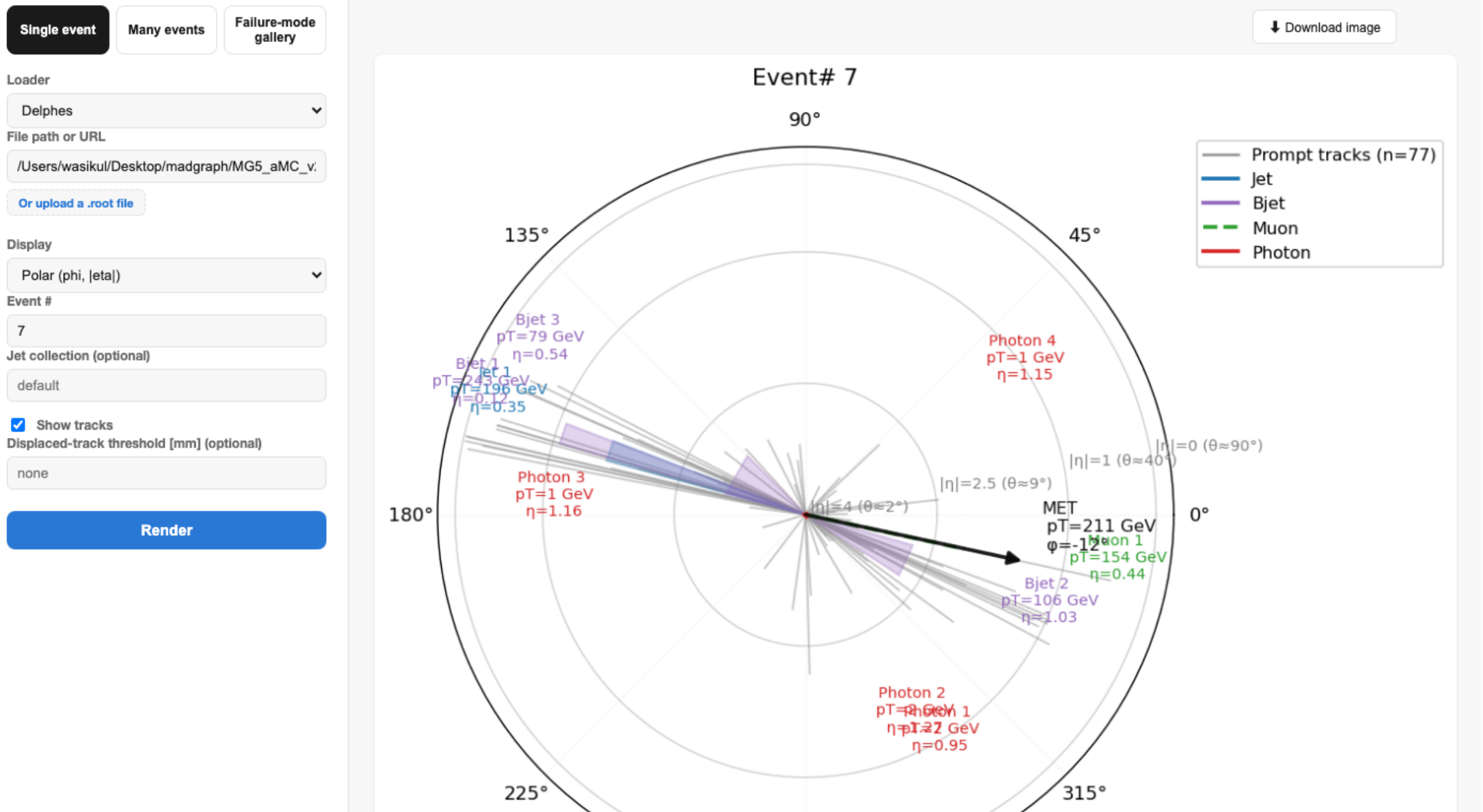
trace-gui: three modes in one page

- * Single event, Many events, Failure-mode gallery -- no script, no pre-rendering
- * Loaders: Delphes, flat-ntuple, ATLAS Open Data, CMS Open Data, and a unified auto-detecting Vertex-display loader
- * Both 2D (matplotlib) and interactive 3D (plotly/WebGL) displays, each with a Download-image button
 - * Every render is a shareable URL (?path=...&loader=...&event_index=...)
 - * One new dependency: Flask, via pip install "trace-hep[gui]"

Single event: pick a loader, it renders live

TRACE Live Viewer

Point at a file, pick a loader and display, type an event number.



```
>>> trace.plot_event_polar(event, show_tracks=True)
```

ttbar Delphes event #7 -- loaded and drawn on demand, with a Download-image button on every render

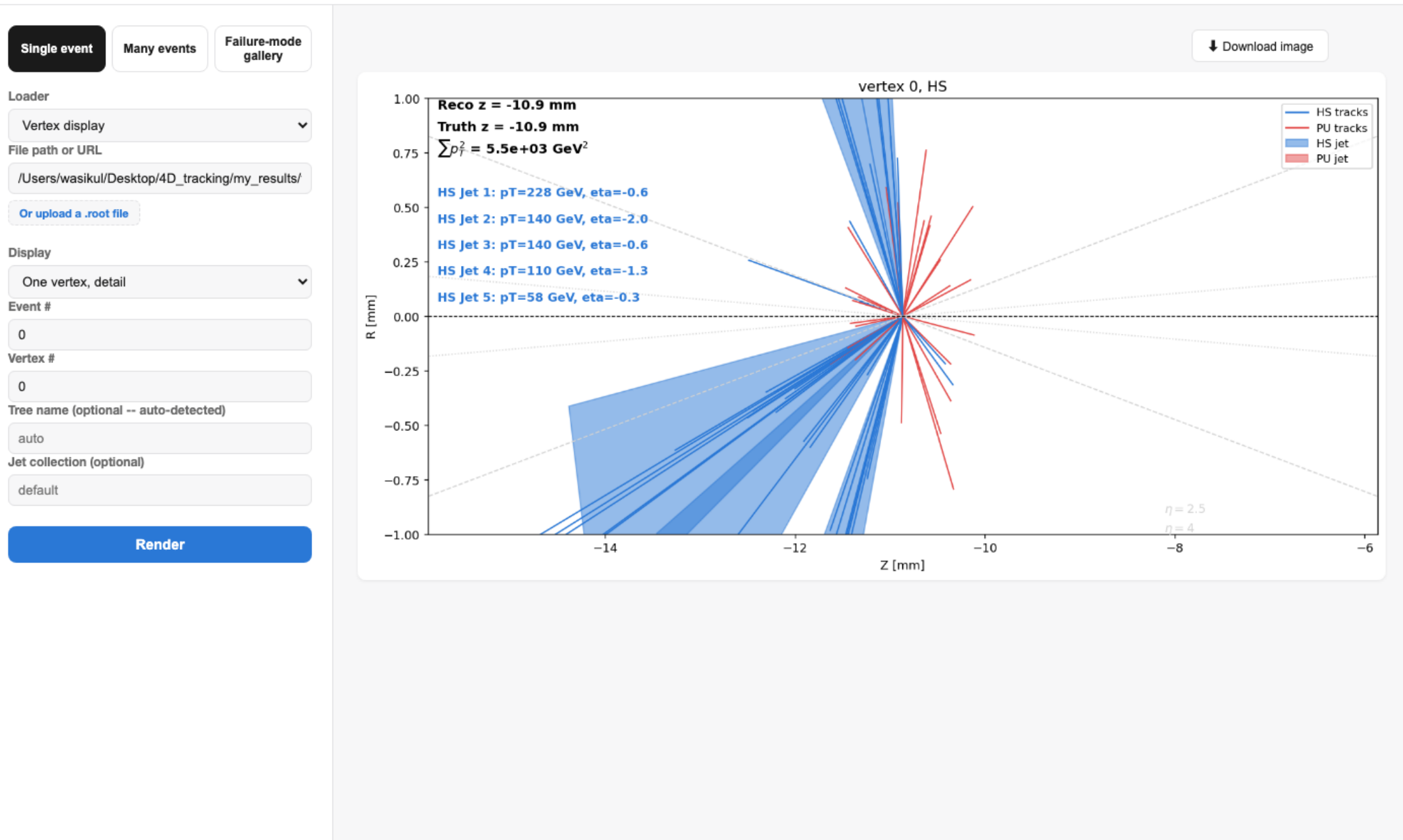
Same event, more views -- one dropdown

- * Display dropdown switches polar <-> beam-axis 2D <-> interactive 3D for the same loaded event
>>> trace.plot_event_beam2d(event) / trace.plot_event_3d(event)
- * "Show tracks" + "Displaced-track threshold (mm)" highlight displaced tracks in any 2D/3D view
>>> trace.plot_event_polar(event, show_tracks=True, d0_displaced_mm=1.0)
- * "Jet collection" field re-reads a different Delphes branch (e.g. ParticleFlowJet04), no code change
>>> load_events(path, indices=[7], jet_collection="ParticleFlowJet04")

Vertex display, auto-detected

TRACE Live Viewer

Point at a file, pick a loader and display, type an event number.



```
>>> trace.plot_vertices_zr(vtx_event, style="styled")
```

A real calo-timing ntuple, picked up automatically -- no format picker needed

Vertex displays: styles, zoom, and detail

```
>>> plot_vertices_zr(vtx_event, style="plain" | "styled" | "time_colored")  
three colour/sizing schemes on the same z-R display (no GUI style picker yet)
```

```
>>> plot_vertices_zr(vtx_event, style="styled", zoom_center=-44.2, zoom_range_mm=5.0)  
zoom to one vertex
```

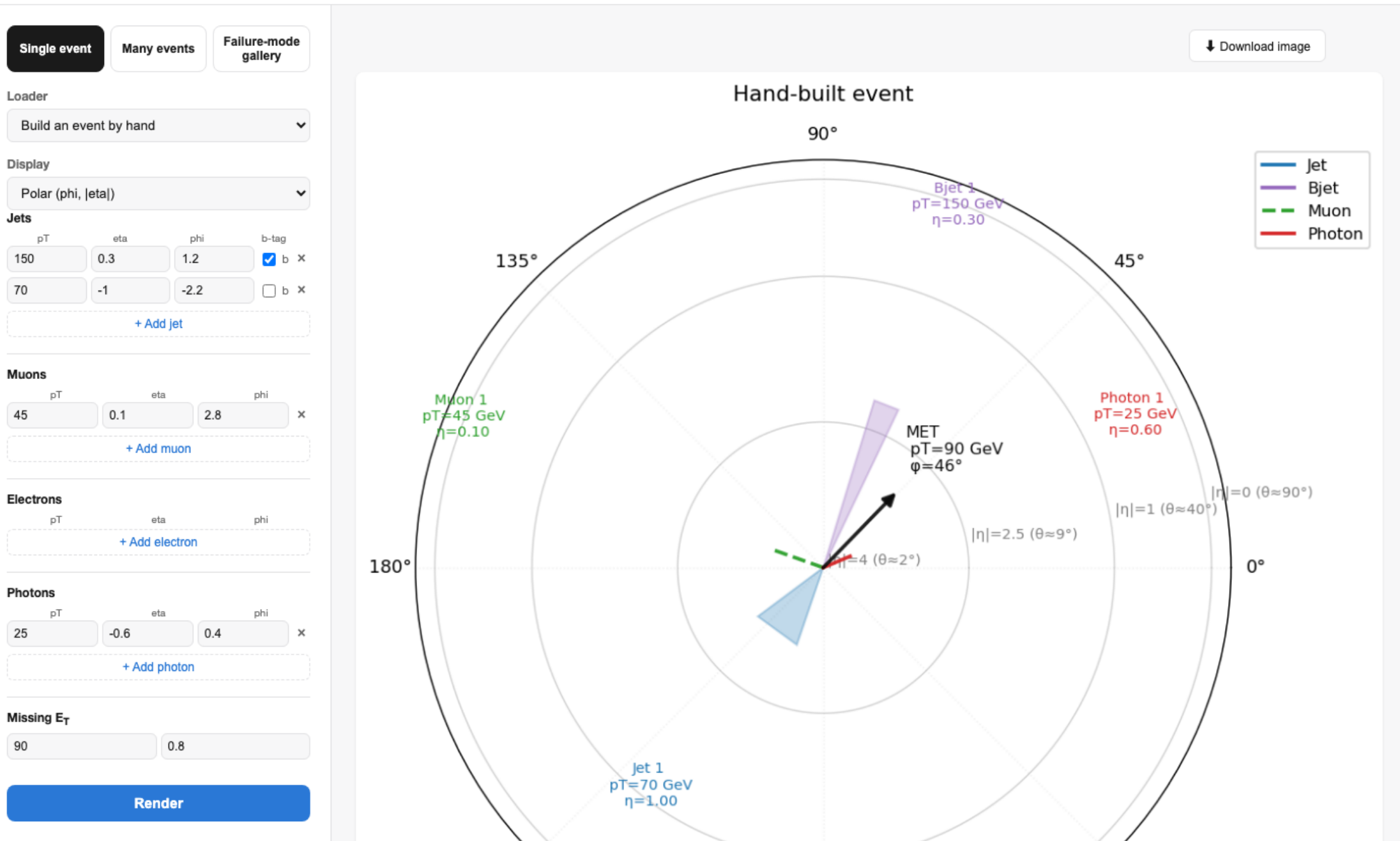
```
>>> plot_vertex_detail(vtx_event, vtx_index, jets=match_jets_to_vertex(...))  
one vertex + its matched jets, tracks/jets coloured by truth match where available
```

```
>>> plot_vertices_3d(vtx_event)  
interactive 3D, every reconstructed and truth vertex at its real (x, y, z)
```

Build an event by hand -- no file needed

TRACE Live Viewer

Point at a file, pick a loader and display, type an event number.



```
>>> trace.Event(jets=[...], muons=[...], met=trace.MissingET(...))
```

2 jets (one b-tagged), 1 muon, 1 photon, MET -- typed directly into the form, no file, no script

Many events -- a plain list, no categories

TRACE Live Viewer

Point at a file, pick a loader and display, type an event number.

Single event

Many events

Failure-mode gallery

Loader

Vertex display

File path or URL

/Users/wasikul/Desktop/4D_tracking/my_results/

Or upload a .root file

Display

Vertices z-R (plain)

Event numbers

0-3

Comma/space/newline-separated; "a-b" expands to an inclusive range.

Run # (flat_ntuple only)

0

Vertex # (vertex display only)

0

Tree name (vertex display, optional -- auto-detected)

auto

Jet collection (delphes / vertex detail, optional)

default

☐ Show tracks (delphes)

☐ Include large-R jets (ATLAS Open Data)

Displaced-track threshold [mm] (delphes, optional)

none

Gallery title

Many events, no categories

Image DPI (lower = smaller/faster for many events)

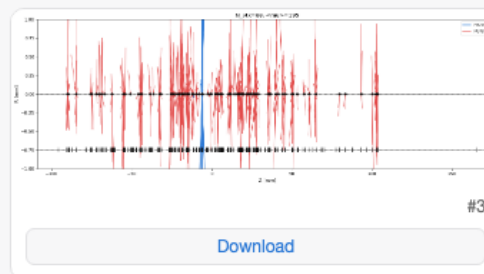
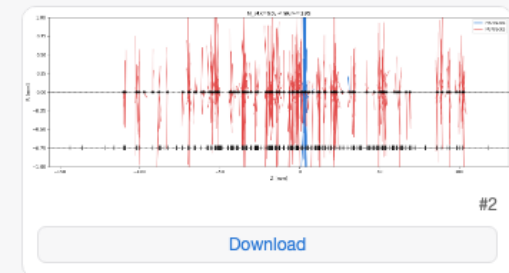
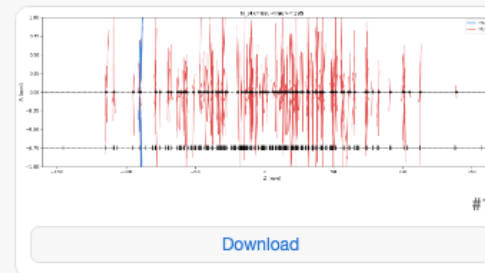
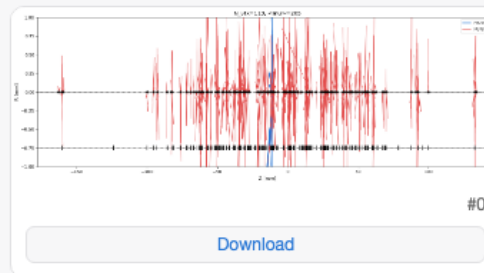
80

Many events, no categories · 4 events

All (4)

Download all (visible)

Jump to event #...



```
>>> run_event_batch(events, "out/", which=("polar", "beam2d"))
```

4 real events from a calo-timing ntuple, just listed as "0-3" -- reviewed without a second file or any pass/fail bookkeeping

Failure-mode gallery -- custom labels

TRACE Live Viewer

Point at a file, pick a loader and display, type an event number.

Single event

Many events

Failure-mode gallery

Loader

Vertex display

File path or URL

/Users/wasikul/Desktop/4D_tracking/my_results/

Or upload a .root file

Display

Vertices z-R (plain)

Run # (flat_ntuple only)

0

Vertex # (vertex display only)

0

Tree name (vertex display, optional -- auto-detected)

auto

Jet collection (delphes / vertex detail, optional)

default

☐ Show tracks (delphes)

☐ Include large-R jets (ATLAS Open Data)

Displaced-track threshold [mm] (delphes, optional)

none

Categorize events by

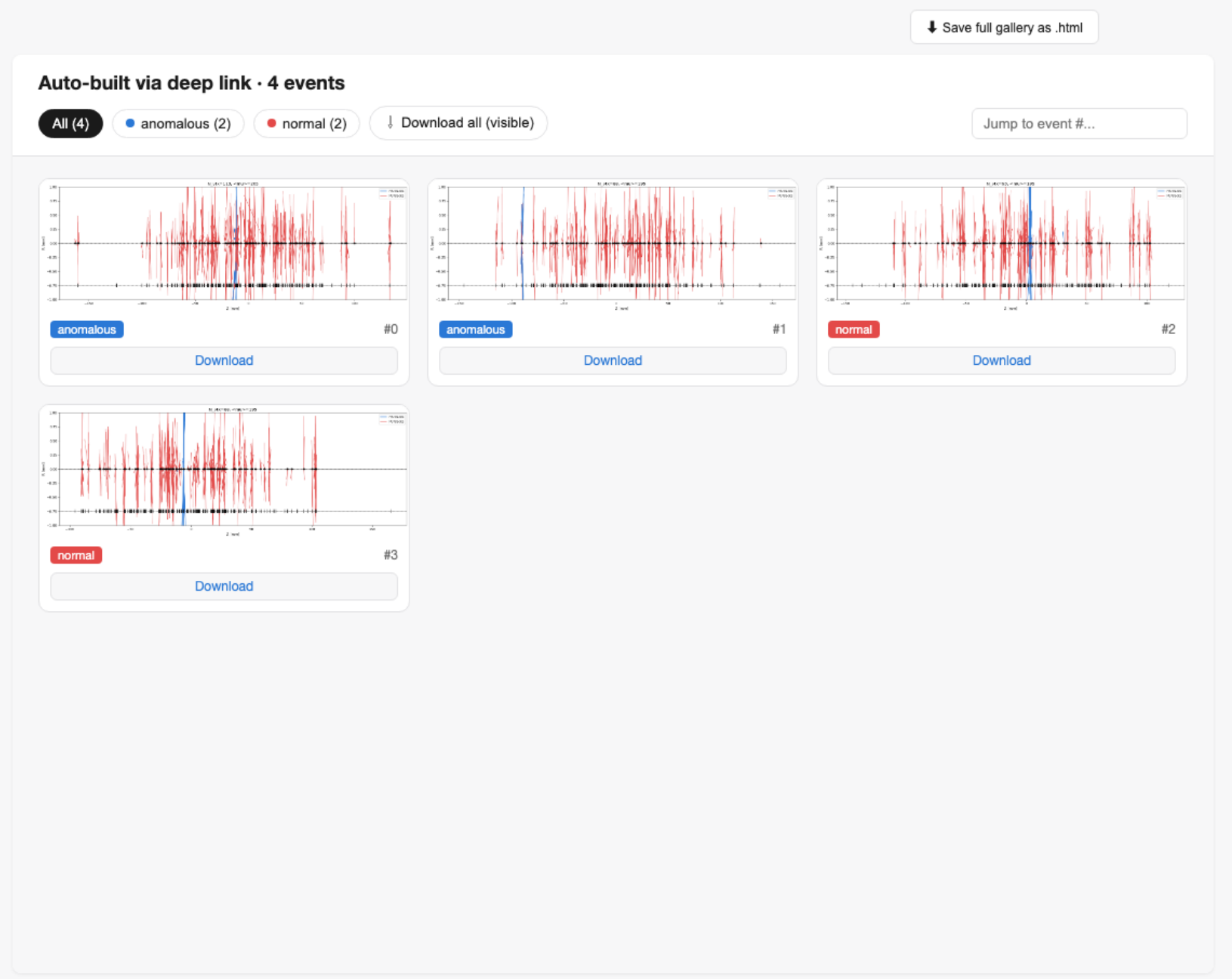
Custom labels

Event id: label (one per line)

0: anomalous
1: anomalous
2: normal
3: normal

Any free-text label works -- every distinct one becomes a filter tab.

Gallery title



```
>>> build_gallery(events, categories, plot_fn, "review.html")
```

4 real events, custom-labeled anomalous/normal, built and reviewed without leaving the browser

Failure-mode gallery -- auto-flag by 11 criteria

TRACE Live Viewer

Point at a file, pick a loader and display, type an event number.

Single event

Many events

Failure-mode gallery

Loader

Delphes

File path or URL

/Users/wasikul/Desktop/madgraph/MG5_aMC_v.

Or upload a .root file

Display

Polar (phi, |eta|)

Run # (flat_ntuple only)

0

Vertex # (vertex display only)

0

Tree name (vertex display, optional -- auto-detected)

auto

Jet collection (delphes / vertex detail, optional)

default

Show tracks (delphes)

Include large-R jets (ATLAS Open Data)

Displaced-track threshold [mm] (delphes, optional)

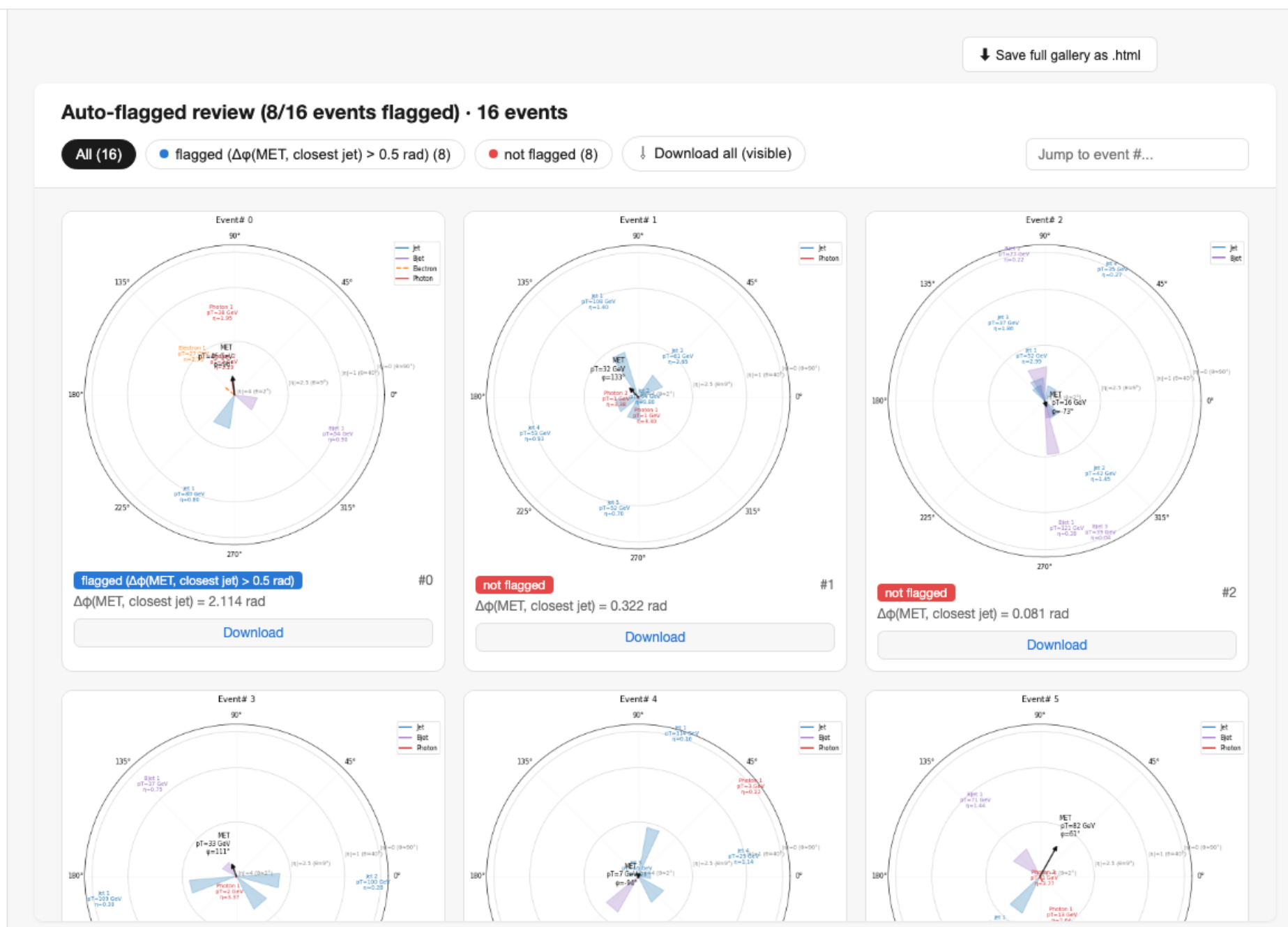
none

Categorize events by

Auto-flag by $\Delta\phi(\text{MET}, \text{closest jet})$

Event range to scan

0-15



16 real $t\bar{t}b\bar{a}r$ Delphes events, auto-flagged by $\Delta\phi(\text{MET}, \text{closest jet}) > 0.5 \text{ rad}$ -- one of 11 built-in criteria (min-Delta-phi among leading jets, MET/HT, MET, HT, jet/b-jet/lepton count, leading-jet pT, dijet back-to-back deviation, max displaced-track $|d_0|$, ...); same mechanism for any of them, just pick from the dropdown

Sharing it: file upload + public hosting

- * Every "File path or URL" field also has an "Or upload a .root file" button -- a remote visitor's browser has no access to a local path, so it uploads the file's bytes instead; everything downstream is unchanged
 - * Uploads are capped at 150 MB and auto-deleted after about an hour
- * `deploy/huggingface-space/` ships a ready-made Docker setup for Hugging Face Spaces' free CPU tier: push it, and colleagues can try their own files at a public URL, no install needed on their end

Real public data, same GUI

- * ATLAS Open Data (13 TeV mini-ntuples) and CMS Open Data (2012 NanoAOD outreach, MC + real collision data) load through the SAME loader dropdown as Delphes -- point at a local file or a URL, click Render
- * Large CMS files stream directly over HTTPS via uproot -- a 3.5 GB file, only the needed bytes are read, nothing downloaded
 - * ATLAS's 2024 PHYSLITE research release adds a real public vertex display too (InDetTrackParticles + PrimaryVertices) -- same "Vertex display" loader, auto-detected; it's thinned, though: only ~3.6% of pileup vertices keep any tracks (measured on 200 events), while the hard-scatter vertex almost always does
- * Finding dataset URLs: `pip install "trace-hep[opendata]" -> atlasopenmagic.get_urls(dataset_id, skim=..., protocol="https")`

Same three examples, as a script

```
# ATLAS Open Data
from tracehep.io.atlas_opendata import load_events
events = load_events("mc_410000.ttbar_lep.2lep.root", indices=[1])

# CMS Open Data, streamed straight from opendata.cern.ch
from tracehep.io.cms_opendata import load_events
url = "https://opendata.cern.ch/eos/opendata/cms/derived-data/.../TTbar.root"
events = load_events(url, indices=[9])

# ATLAS PHYSLITE, real public vertex + track data
from tracehep.io.atlas_physlite import load_vertex_event
vtx_event = load_vertex_event(url, event_index=50)
```

Every one of these renders through the same `trace.plot_event_polar / plot_vertices_zr` calls used everywhere else in this deck -- and through the same GUI loader dropdown.

Filtering, in scripts and from the CLI

Jet collection is a loader choice -- the GUI's "Jet collection" field, or `jet_collection=...` when loading (see the vertex-display slide above)

```
>>> filter_event(event, jet_pt_min=50.0, jet_eta_min=-2.5, jet_eta_max=2.5, track_pt_min=2.0, ...)
      pT/eta cuts, script-side, not yet exposed as GUI fields
>>> filter_vertex_event(vtx_event, track_pt_min=1.0, track_eta_min=-2.5, track_eta_max=2.5)
      same idea for a vertex scenario -- vertices are never dropped, only the tracks fit to them
```

\$ `trace-batch --input sample.root --jet-pt-min 30 --jet-eta-min -2.5 --jet-eta-max 2.5 --show-tracks`

every filter is also a trace-batch flag -- no code needed

Batch processing many events at once

```
from tracehep.io.delphes import load_events
from tracehep.batch import run_event_batch, run_vertex_batch

events = load_events(path, indices=range(20))
run_event_batch(events, "out/", which=("polar", "beam2d"))
```

```
# or, from the shell, no Python needed:
trace-batch --input file.root --indices 0 1 2 --outdir out/ --which polar beam2d
```

run_vertex_batch(vertex_events, outdir, styles=(...)) is the vertex-scenario equivalent.

Function reference: displays

```
plot_event_polar(event, *, show_tracks=False, d0_displaced_mm=None, title=None, ...)
    plot_event_beam2d(event, *, angle_scale=4.0, title=None, ...)
    plot_event_3d(event, *, d0_displaced_mm=None, title=None, ...)

plot_vertices_zr(vertex_event, *, style="plain", zoom_center=None, zoom_range_mm=None,
                  title=None)
plot_vertex_detail(vertex_event, vtx_index, jets=(), *, zoom_range_mm=5.0, title=None)
    plot_vertices_3d(vertex_event, *, title=None)

run_event_batch(events, outdir, *, which=(...), polar_kwargs=None, beam2d_kwargs=None)
    run_vertex_batch(vertex_events, outdir, *, styles=(...))
```

Loader reference (tracehep.io) 1/2

```
delphes.load_event(s)(path, index(es), with_tracks=True, jet_collection="Jet", label="")
```

```
flat_ntuple.load_event(s)_by_run_event(path, run, event, tree_name="Ntuple",  
                                       jet_branches=(...), bjet_branches=(...), label="")
```

```
calotiming.load_vertex_event(path, event_index, tree_name="ntuple", label="")  
    calotiming.match_jets_to_vertex(path, event_index, vtx_z, *,  
    jet_collection="AntiKt4EMTopoJets", jet_pt_min=30.0, rpt_min=0.02, sig_cut=3.0)
```

All four require: `pip install "trace-hep[delphes]"`

Loader reference (tracehep.io) 2/2

```
        atlas_opendata.load_event(s)(path, index(es), tree_name="mini",
                                     btag_cut=0.6459, include_large_r_jets=False, label="")
-- ATLAS Open Data 13 TeV mini-ntuples; object-level only, no tracks/vertices

cms_opendata.load_event(s)(path, index(es), tree_name="Events", btag_cut=0.679, label="")
-- CMS Open Data reduced NanoAOD, 2012 8 TeV, MC and real collision data;
   path can be a local file or an opendata.cern.ch URL (streamed)

atlas_physlite.load_vertex_event(path, event_index, tree_name="CollectionTree", label="")
   atlas_physlite.load_event_jets(path, event_index, jet_pt_min=20.0)
-- ATLAS's 2024 PHYSLITE research release; jets carry no vertex association/truth
```

Requires: `pip install "trace-hep[delphes]"` (+ `[opendata]` for URL discovery)

Filter & gallery reference

```
filter_event(event, *, jet_pt_min=None, jet_pt_max=None, jet_eta_min=None, jet_eta_max=None,
             track_pt_min=None, track_pt_max=None, track_eta_min=None, track_eta_max=None)
filter_vertex_event(vertex_event, *, track_pt_min=None, ..., track_eta_max=None)
filter_jets(jets, *, pt_min=None, pt_max=None, eta_min=None, eta_max=None, btag_only=False)
filter_tracks(tracks, *, pt_min=None, pt_max=None, eta_min=None, eta_max=None)
    -- all return a filtered COPY, input never modified

build_gallery(events, categories, plot_fn, output_path, *, title="...", dpi=100, captions=None)
    compare_pass_fail(results_a, results_b, *, name_a="algo1", name_b="algo2")
    -- dpi: drop to 60-80 for hundreds-thousands of events to keep the HTML file responsive
```


Viewer reference (tracehep.webapp)

```
trace-gui [--host 127.0.0.1] [--port 5057] [--no-browser]  
python3 -m tracehep.webapp (equivalent)
```

`create_app()` -- returns the Flask app undecorated, for embedding or testing
(e.g. `app.test_client()`) without starting a server

URL query params pre-fill the form and auto-render; `?mode=many` and
`?mode=gallery` deep-link straight into the two gallery tabs

`/api/upload` accepts a multipart .root file ($\leq 150\text{MB}$), returns a server-side
path; a background thread deletes uploads after $\sim 1\text{hr}$

Requires: `pip install "trace-hep[gui]"` (adds Flask)

Troubleshooting

- * ImportError: uproot required
-> pip install "trace-hep[delphes]"
- * ModuleNotFoundError: No module named tracehep (right after install)
 - > from TestPyPI you need BOTH --index-url and --extra-index-url (see Installation slides)
- * KeyError from flat_ntuple loader
-> the (run, event) pair is not present in that file
- * A track's 3D origin looks wrong from a calo-timing ntuple
 - > that format has no per-track (x, y, z); origin defaults to the owning vertex's (x, y) plus the track's own z0

Versioning and updates

- * Published package versions are immutable

- > updating requires bumping the version and re-uploading

- * Typical update loop:

- edit code in src/tracehep/

- add/update a test

- bump version in pyproject.toml and __init__.py

- python3 -m build

- twine upload --repository-url <https://test.pypi.org/legacy/> dist/*

Get involved

Source: `github.com/wasikatcern/TRACE-HEP`
Install: `pip install trace-hep` (TestPyPI now; PyPI planned)
License: MIT
Tests: `pytest -q` (94 passing at v0.1.19)
Contact: `wasikul.islam@cern.ch`

Citing: accompanying paper citation to be added once posted.